Software Engineering Lab

Lab 22

N+1 Query Problem

Batching Relasi Data untuk Eliminasi Latensi Tersembunyi



Lukman / lukman-ss

Satu Query Awal, Puluhan Query Mengikuti

Pengambilan data berulang di dalam iterasi melipatgandakan latensi jaringan.

MOTIVASI

Perilaku cepat di local development menipu engineer saat masuk ke beban production.
Setiap query individu terlihat sub-milidetik, sehingga lolos dari slow query logs.
Degradasi akumuler: request yang seharusnya 20ms membengkak menjadi beberapa detik.

BUKTI LAB

Store query counter mencatat 1 query author + 3 query post = 4 query total pada 3 data.
Eager loading memangkas eksekusi menjadi 1 query author + 1 query post = 2 query total.

BATASAN SISTEM

Lab memodelkan in-memory Store; pada RDBMS nyata overhead socket TCP memperparah bottleneck.

N+1 query problem merupakan anti-pattern performa paling umum pada data access layer karena sifatnya yang tidak kasat mata pada volume data kecil. Ketika backend mengambil daftar entitas induk lalu menerbitkan query anak di dalam loop perulangan, jumlah round-trip jaringan meningkat secara linear O(N). Pada tahap development dengan segelintir data, eksekusi selesai sub-milidetik. Namun begitu memasuki lingkungan production dengan ratusan record, akumulasi latensi I/O dan perebutan connection pool akan langsung melumpuhkan throughput layanan.

N+1 is invisible until it hurts. Batch related queries, don't loop them.

Arsitektur Tiga Layer Lab 22

Store terisolasi, Repository data access, dan executable demo runner.

Komponen	Peran	File Sumber
Store	Mock database thread-safe & query counter	internal/blog/store.go
Repository	Logika query Naive loop vs Eager batching	internal/blog/repository.go
Demo	Eksekusi CLI & verifikasi statistik I/O	cmd/demo/main.go

Arsitektur Lab 22 dibangun dengan pemisahan tanggung jawab yang ketat untuk mengisolasi pengukuran interaksi database. Paket Store bertindak sebagai mock database in-memory yang mencatat setiap operasi baca secara atomik melalui query counter thread-safe. Lapisan Repository mengonsumsi Store untuk menyediakan dua variasi endpoint data access: implementasi naive yang mengeksekusi query di dalam loop, serta implementasi eager yang melakukan batching via filter multi-ID. Pemisahan ini memungkinkan pengujian matematis yang deterministik tanpa dependensi eksternal.

Clean layer isolation enables precise query instrumentation and verification.

Entitas Domain: Author dan Post

Relasi satu-ke-banyak (one-to-many) antara penulis blog dan artikelnya.

```
type Author struct {
    ID   int
    Name string
}

type Post struct {
    ID       int
    AuthorID int
    Title    string
}

type AuthorWithPosts struct {
    Author Author
    Posts  []Post
}
```

Model domain merepresentasikan kasus klasik relational data: satu Author memiliki banyak entitas Post melalui referensi AuthorID. Struktur komposit AuthorWithPosts menjadi agregat yang harus dikembalikan kepada consumer API. Pada relasi one-to-many seperti ini, keputusan arsitektural apakah relasi child diambil secara lazily per item atau secara batched akan menentukan total beban jaringan antara service backend dan database engine.

Aggregate roots must balance relation completeness with network round-trips.

Loop-Driven Data Retrieval

Mengambil data relasi child satu per satu di dalam perulangan entitas parent.

```
func (r *Repository) GetAuthorsWithPostsNPlusOne() []AuthorWithPosts {
    authors := r.store.GetAllAuthors()
    if len(authors) == 0 { return []AuthorWithPosts{} }

    var result []AuthorWithPosts
    for _, author := range authors {
        posts := r.store.GetPostsByAuthorID(author.ID)
        result = append(result, AuthorWithPosts{
            Author: author,
            Posts:  posts,
        })
    }
    return result
}
```

Implementasi naive pada GetAuthorsWithPostsNPlusOne memperlihatkan bagaimana kode yang tampak bersih dan intuitif dapat menyembunyikan bencana skalabilitas. Pertama, metode memanggil GetAllAuthors yang menghasilkan 1 query. Selanjutnya, perulangan for mengiterasi setiap author dan memanggil GetPostsByAuthorID secara individual. Jika terdapat N author, sistem memicu N query tambahan sehingga total query yang dieksekusi menjadi 1 + N. Pola loop-query ini sangat sering dihasilkan secara otomatis oleh mekanisme lazy loading bawaan ORM.

Querying inside loops turns iteration into cumulative network latency.

Mengapa Slow Query Log Gagal Mendeteksi

Setiap query individual sangat cepat, tetapi total durasi request meledak.

TITIK BUTA DETEKSI

Slow query log hanya menandai eksekusi dengan durasi di atas ambang batas (mis. >1000ms).
Query N+1 masing-masing berjalan sub-milidetik (<1ms) karena memakai primary/foreign key index.
Database engine menganggap sistem bekerja normal tanpa anomali query berat.

AKUMULASI WAKTU

Development: 3 author memicu 4 query, selesai dalam tempo <5ms.
Production: 500 author memicu 501 query, menghabiskan waktu 2.4 detik.
Beban berpindah ke latency transfer jaringan dan serialisasi paket di aplikasi.

Mekanisme pemantauan database standar seperti slow query log dirancang untuk menangkap eksekusi query tunggal yang tidak efisien—misalnya akibat full table scan atau hilangnya index. Namun, query pada N+1 biasanya berupa pencarian sederhana berbasis index (WHERE author_id = ?) yang selesai dalam waktu 0.5 milidetik. Akibatnya, database engine mencatat performa normal tanpa ada alert. Beban justru terakumulasi pada latensi perjalanan paket bolak-balik di jaringan dan serialisasi data di application server.

Sub-millisecond individual queries can aggregate into multi-second latency.

Biaya Tersembunyi: Network Round-Trip Time

Overhead protokol, packet traversal, dan context switching aplikasi.

Komponen Biaya	1 Query Batch	100 Query N+1
Network RTT	1 x 2ms = 2ms	100 x 2ms = 200ms
TCP Wire Overhead	1 frame exchange	100 frame exchanges
DB Engine Parsing	1× AST parse	100× AST parse
App Context Switch	Minimal	Sangat tinggi (100× I/O)

Ketika aplikasi berkomunikasi dengan database melalui jaringan TCP, setiap query harus menanggung biaya Round-Trip Time (RTT), handshake protocol packet, context switching, serta parsing SQL abstract syntax tree di sisi database. Pada topologi cloud modern di mana database berada di server terpisah, RTT tipikal berkisar antara 1 hingga 5 milidetik. Menjalankan 100 query terpisah berarti membuang minimal 200 hingga 500 milidetik murni hanya untuk waktu transit paket jaringan, terlepas dari seberapa cepat mesin database mengeksekusinya.

Network round-trips dominate relational latency over raw query execution.

Bahaya Connection Pool Exhaustion

Rentetan query berurutan menahan koneksi dan memicu antrean thread backend.

SEQUENTIAL HOLD

Satu HTTP worker menahan koneksi database selama seluruh siklus iterasi loop N.
Koneksi tidak dapat dipakai oleh worker lain hingga iterasi terakhir selesai.

POOL STARVATION

Kenaikan concurrency membuat kapasitas connection pool langsung habis (exhausted).
Request baru mengalami blocking queue di pool dan berakhir dengan gateway timeout (504).

CASCADING FAILURE

Satu endpoint lambat melumpuhkan seluruh service yang berbagi connection pool database.

Dampak sistemik paling destruktif dari N+1 query problem adalah pengurasan connection pool (connection pool starvation). Dalam arsitektur pooling standar, satu thread web request meminjam koneksi database dan menahannya selama iterasi berlangsung. Jika 50 request konkuren masing-masing membutuhkan 100 query berurutan, pool database yang berkapasitas 20 koneksi akan langsung tersaturasi. Request lain yang membutuhkan query kilat 2 milidetik terpaksa mengantre di queue pool hingga timeout, menyebabkan kegagalan berantai di seluruh layanan.

Sequential queries monopolize connection pools and trigger cascading timeouts.

Solusi Eager Loading: Pola Batch Data Retrieval

Ekstraksi seluruh identifier relasi sebelum mengirim satu query gabungan.

ALUR 4 TAHAP EAGER BATCHING

```
TAHAP 1: Ambil Parent —— GetAllAuthors() → [ID: 1, 2, 3] (Query #1)
TAHAP 2: Ekstraksi Kunci —— authorIDs = [1, 2, 3] di memori
TAHAP 3: Ambil Child Sekaligus —— GetPostsByAuthorIDs([1, 2, 3]) (Query #2)
TAHAP 4: Asosiasi Memori —— Kelompokkan posts ke author via map[int]
[]Post
```

Prinsip utama penyelesaian N+1 adalah memisahkan fase pengumpulan kebutuhan data dari fase eksekusi query database. Pendekatan eager batching membaca seluruh parent record terlebih dahulu, mengekstrak seluruh primary key yang relevan ke dalam slice memori, lalu menerbitkan satu query tunggal untuk mengambil seluruh child record yang cocok. Dengan pola ini, terlepas dari apakah terdapat 3, 50, atau 1000 author, jumlah interaksi database dipangkas secara drastis menjadi tepat dua query I/O.

Batch before you iterate: decouple query planning from data association.

Query Batching di Sisi Store

Memanfaatkan hash lookup dan filter multi-ID untuk mengambil child records.

```
func (s *Store) GetPostsByAuthorIDs(authorIDs []int) []Post {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.queryCount++

    idMap := make(map[int]bool)
    for _, id := range authorIDs { idMap[id] = true }

    var result []Post
    for _, p := range s.posts {
        if idMap[p.AuthorID] {
            result = append(result, p)
        }
    }
    return result
}
```

Di level Store, metode GetPostsByAuthorIDs merepresentasikan eksekusi klausa SQL WHERE author_id IN (...). Implementasi in-memory membangun idMap untuk lookup cepat bertipe O(1) sebelum memfilter koleksi posts. Yang paling krusial: fungsi ini hanya menaikkan counter queryCount sebanyak tepat 1 kali untuk seluruh daftar authorIDs. Di sistem produksi, operasi ini diterjemahkan menjadi satu perintah SQL dengan transmisi payload tunggal, menghemat 99% overhead protokol database.

A single set-based filter replaces hundreds of point lookups.

In-Memory Map Grouping pada Layer Aplikasi

Mengelompokkan hasil batch ke dalam hash map untuk asosiasi instan O(1).

```
// 1. Kelompokkan posts berdasarkan author ID
postsByAuthor := make(map[int][]Post)
for _, post := range allPosts {
    postsByAuthor[post.AuthorID] = append(
        postsByAuthor[post.AuthorID], post)
}

// 2. Susun agregat akhir dengan pencarian O(1)
var result []AuthorWithPosts
for _, author := range authors {
    result = append(result, AuthorWithPosts{
        Author: author,
        Posts:  postsByAuthor[author.ID],
    })
}
```

Tantangan setelah mengambil sekumpulan child records adalah merekonstruksi relasi hierarkis ke entitas parent aslinya. Solusi optimal di level aplikasi adalah mengelompokkan hasil batch ke dalam tabel hash lokal map[int][]Post dengan key berupa foreign key AuthorID. Iterasi rekonstruksi kemudian hanya membutuhkan pencarian hash O(1) untuk menyematkan slice posts ke masing-masing author. Menghindari nested loop di memori menjamin kompleksitas waktu pemrosesan aplikasi tetap linear O(N + M).

Group child entities in memory to preserve O(1) relational association.

internal/blog/repository.go:45-56

Komparasi Kuantitatif Kompleksitas Sistem

Transformasi kompleksitas I/O dari ketergantungan data O(N) ke deterministik O(1).

Parameter	Naive (Loop)	Eager (Batch)
Query Count	1 + N (Linear)	1 + 1 (Konstan 2)
Kompleksitas I/O	O(N) relatif record	O(1) relatif record
Data Transfer	Header repetitif	Single packet payload
Memory Allocation	Fragmented allocations	Single pre-sized buffer
Network RTT	N + 1 round-trips	2 round-trips

Tabel komparasi menggarisbawahi perubahan fundamental pada profil operasional sistem. Dalam pendekatan loop-driven, kompleksitas I/O bersifat O(N), yang berarti setiap penambahan data author di database secara langsung meningkatkan beban I/O jaringan. Sebaliknya, pendekatan eager loading menurunkan kompleksitas interaksi database menjadi O(1) relatif terhadap jumlah baris data—hanya bertambah jika terdapat entitas relasi tambahan (O(k) di mana k adalah jumlah entitas relasional). Ini menghasilkan stabilitas throughput yang terprediksi.

Shift complexity from database network I/O into deterministic in-memory grouping.

SQL Translation: IN Clause vs Single JOIN

Membandingkan strategi batch query terpisah versus query tunggal dengan JOIN.

```
-- Strategi 1: Batching (Dua Query Terpisah)
SELECT id, name FROM authors;
SELECT id, author_id, title FROM posts
WHERE author_id IN (1, 2, 3);

-- Strategi 2: Single JOIN (Satu Query Tunggal)
SELECT a.id, a.name, p.id, p.title
FROM authors a
LEFT JOIN posts p ON a.id = p.author_id;
```

Dalam dunia RDBMS, terdapat dua pendekatan utama untuk eager loading: dua query terpisah menggunakan klausa IN (split queries), atau satu query tunggal menggunakan LEFT JOIN. Meskipun single JOIN terdengar menggoda karena hanya membutuhkan 1 query, teknik ini menduplikasi data kolom parent pada setiap baris hasil child yang dikembalikan. Pada relasi one-to-many dengan kolom teks besar, duplikasi ini memboroskan bandwidth jaringan. Split queries via IN clause umumnya menjadi pilihan default ORM modern karena lebih efisien terhadap memori.

Split batch queries avoid parent column duplication inherent in wide JOINs.

Batasan Parameter IN Clause pada RDBMS

Batas maksimum parameter dan overhead parsing abstract syntax tree database.

BATASAN PARAMETER

PostgreSQL & SQLite memiliki limit bind parameter (mis. 65,535 atau 999 parameter).
Melebihi batas ini menyebabkan error query execution langsung dari driver database.

OVERHEAD PARSER

Statement SQL dengan ribuan literal ID membebani query planner dan parser CPU.
Waktu penyusunan AST di engine database dapat melebihi waktu eksekusi indeks.

CHUNKED BATCHING

Pecah daftar ID besar menjadi chunk terkendali (misal 200–500 ID per batch query).

Batching menggunakan klausa IN bukan tanpa batas. Database engine memiliki batasan terhadap jumlah parameter bind dalam satu statement (misalnya batas 65.535 parameter pada PostgreSQL atau 999 pada SQLite versi terdahulu). Lebih jauh lagi, statement SQL dengan ribuan literal parameter memaksa query planner menghabiskan waktu CPU berharga hanya untuk mem-parsing sintaks string query. Oleh karena itu, pada skala data masif, aplikasi wajib menerapkan teknik chunked batching—memecah daftar ID menjadi batch terkendali (misal 500 ID per batch).

Unbounded IN clauses choke SQL parsers: chunk bulk IDs into manageable sizes.

Bahaya Cartesian Product Explosion

Risiko ekspansi baris berlipat ganda saat melakukan eager JOIN multi-relasi.

Tingkat Relasi	Entitas Record	Total Baris Hasil JOIN
1 Author	1 author	1 baris
10 Posts	10 post per author	1 x 10 = 10 baris
5 Tags	5 tag per post	1 x 10 x 5 = 50 baris
10 Comments	10 komentar per post	1 x 10 x 5 x 10 = 500 baris!

Ketika pengembang mencoba mengoptimalkan beberapa relasi one-to-many sekaligus (misalnya mengambil Author beserta Posts dan Tags-nya) menggunakan JOIN tunggal, sistem akan terkena fenomena Cartesian Product Explosion. Database mengalikan setiap entitas parent dengan seluruh permutasi child records-nya. Data yang dikembalikan melalui jaringan membengkak ribuan kali lipat dan berpotensi memicu Out-Of-Memory (OOM) pada backend. Solusi arsitektural yang aman adalah membagi relasi menjadi batch queries independen terpisah.

Multiple relational JOINs cause combinatorial row explosions; use separate batches.

Mapping-Level Eager vs Query-Level Eager

Bahaya anotasi global FetchType.EAGER dibandingkan selective fetching.

MAPPING-LEVEL (ANTI-PATTERN)

FetchType.EAGER pada schema entity menarik seluruh relasi setiap kali entitas dibaca.
Operasi sederhana seperti validasi email author terpaksa memuat seluruh riwayat postingan.
Menimbulkan memory bloat parah dan membebani garbage collector aplikasi.

QUERY-LEVEL (BEST PRACTICE)

Relasi dibiarkan lazy secara default pada level definisi domain model.
Eager loading hanya diaktifkan secara eksplisit pada query yang benar-benar membutuhkannya.

Kesalahan umum yang sering terjadi di level framework (khususnya JPA/Hibernate) adalah menerapkan mapping-level eager loading (FetchType.EAGER) secara global pada entity schema. Hal ini menyebabkan setiap pembacaan entitas Author otomatis menarik seluruh Posts, Comments, dan Metadata terkait di mana pun dipanggil. Pendekatan profesional menuntut query-level eager loading: biarkan relasi default terisolasi, dan aktifkan batching hanya secara eksplisit pada lapisan repository atau service yang benar-benar membutuhkan agregat data tersebut.

Enforce query-level eager fetching; avoid global schema-level eager mappings.

Solusi Ekosistem: GraphQL DataLoader Pattern

Mengotomatisasi penundaan dan batching query pada resolver independen.

ALUR RESOLVER COALESCING



Pada arsitektur GraphQL, masalah N+1 muncul secara alami karena setiap field resolver dieksekusi secara independen. Filosofi batching diwujudkan melalui pola DataLoader. DataLoader bekerja dengan menampung (coalescing) permintaan fetch ID individual yang terjadi dalam satu event loop tick, menunda eksekusi ke database, lalu mengirimkan satu query batch terkonsolidasi. DataLoader membuktikan bahwa prinsip pemisahan pengumpulan kunci dari eksekusi I/O dapat diterapkan secara elegan pada lapisan middleware.

DataLoaders coalesce concurrent point queries across async resolver ticks.

Column Selection & Aggregation Optimization

Memilih kolom spesifik atau kalkulasi database saat entitas penuh tidak diperlukan.

```
-- 1. Ambil ringkasan agregasi alih-alih seluruh baris post
SELECT author_id, COUNT(*) AS post_count
FROM posts WHERE author_id IN (1, 2, 3)
GROUP BY author_id;

-- 2. Proyeksikan kolom esensial saja
SELECT id, author_id, title
FROM posts WHERE author_id IN (1, 2, 3);
```

Eager loading penuh tidak selalu menjadi solusi terbaik jika kebutuhan bisnis hanyalah informasi ringkasan atau segelintir kolom. Jika use-case hanya ingin menampilkan jumlah artikel tiap penulis, memuat seluruh objek Post ke dalam memori aplikasi adalah pemborosan besar. Eksekusi agregasi langsung di database melalui GROUP BY atau subquery scalar menghasilkan efisiensi transfer data yang jauh lebih superior, menjaga footprint RAM aplikasi tetap ramping.

Push aggregation to the database engine before pulling bulk entities into memory.

Demo CLI Execution Output

Bukti eksekusi nyata dari cmd/demo/main.go pada dataset 3 author.

```
$ go run ./cmd/demo
--- 1. Simulating N+1 Query Problem ---
Loaded 3 authors with their posts.
Total queries executed: 4 (1 for authors + 3 queries for posts)

--- 2. Simulating Eager Loading (Batching) ---
Loaded 3 authors with their posts.
Total queries executed: 2 (1 for authors + 1 batched query for posts)
```

Eksekusi demo pada cmd/demo/main.go mendemonstrasikan kontras performa secara langsung di terminal console. Pada dataset uji dengan 3 author (Alice, Bob, Charlie) dan 5 total post, implementasi naive mencatat tepat 4 query database. Sebaliknya, implementasi eager loading menyelesaikan seluruh pemuatan agregat dengan tepat 2 query. Verifikasi langsung ini menegaskan validitas teori tanpa bergantung pada konfigurasi database pihak ketiga yang rumit.

Empirical CLI verification validates algorithmic query reduction.

Pengujian Otomatis Query Count Invariant

Memverifikasi jumlah query secara deterministik dalam unit test CI/CD.

```
func TestGetAuthorsWithPostsNPlusOne(t *testing.T) {
    store := NewStore()
    repo := NewRepository(store)
    store.ResetQueryCount()

    result := repo.GetAuthorsWithPostsNPlusOne()
    if len(result) != 3 { t.Fatalf("expected 3, got %d", len(result)) }

    queryCount := store.GetQueryCount()
    if queryCount != 4 {
        t.Errorf("expected 4 queries (N+1), got %d", queryCount)
    }
}
```

Salah satu tantangan terbesar N+1 adalah kerentanannya terhadap regresi kode di masa depan: seorang engineer baru dapat secara tidak sengaja menambahkan pemanggilan accessor di dalam template atau loop. Lab 22 menunjukkan bagaimana menulis unit test yang memverifikasi query count secara deterministik. Dengan menginstruksikan Store mock untuk menghitung pemanggilan query, pipeline CI/CD dapat secara otomatis menggagalkan build jika jumlah interaksi database melebihi ambang batas invarian.

Lock query count invariants into CI tests to permanently prevent regressions.

Validasi Kesetaraan Semantik (Deep Equivalence)

Memastikan optimasi query tidak merusak struktur atau kebenaran data bisnis.

```
// Memastikan hasil batching 100% identik dengan naive
nPlusOneResult := repo.GetAuthorsWithPostsNPlusOne()
if !reflect.DeepEqual(result, nPlusOneResult) {
    t.Errorf("eager result mismatch with N+1 result:\n"+
        "Eager: %+v\nN+1: %+v", result, nPlusOneResult)
}
```

Optimasi performa tidak ada artinya jika mengubah integritas atau susunan data yang diterima oleh pemanggil. Dalam TestGetAuthorsWithPostsEager, paket menguji bahwa slice AuthorWithPosts yang dihasilkan oleh algoritma batching identik secara mendalam (deeply equal) dengan hasil algoritma naive menggunakan reflect.DeepEqual. Pengujian ini membuktikan bahwa pengelompokan memori berbasis hash map mempertahankan struktur data bisnis secara sempurna tanpa efek samping.

Performance refactoring must guarantee semantic data equivalence.

internal/blog/repository_test.go:45–50

Observability & Deteksi N+1 di Production

Instrumentasi OpenTelemetry spans, query profiling, dan static linters.

DISTRIBUTED TRACING

APM trace menampilkan 'waterfall graph' berisi puluhan span query identik berurutan.
Waktu tunggu jaringan antar span memperjelas latensi akumulatif yang dialami client.

MIDDLEWARE AUDITING

Pasang query logger di database driver untuk mendeteksi repetisi query template per HTTP request.
Beri threshold alarm jika satu request mengeksekusi >20 query serupa dalam durasi <50ms.

STATIC ANALYSIS

Gunakan linter kode untuk menandai eksekusi database calls di dalam perulangan loop for.

Di lingkungan produksi skala besar, deteksi N+1 dilakukan melalui instrumentasi Application Performance Monitoring (APM) dan OpenTelemetry distributed tracing. Trace visual memperlihatkan serangkaian span database berurutan dengan query template yang sama persis (waterfall trace). Melengkapi tracing dengan query counter middleware dan static code analysis linter memastikan bahwa setiap anomali rasio query terhadap HTTP request langsung memicu notifikasi peringatan sebelum mengganggu ketersediaan sistem.

Waterfall APM traces and query-to-request ratios surface hidden runtime loops.

Reproduksi Mandiri & Panduan Verifikasi

Seluruh implementasi berbasis Go standard library siap dijalankan dan diuji.

```
cd labs/22-n-plus-one-query-problem
go test -v ./...
go test -race ./...
go run ./cmd/demo
```

Modul Lab 22 ditulis menggunakan Go 1.22 standar tanpa framework eksternal, memastikan portabilitas tinggi dan reproduksi instan di mesin apa pun. Perintah go test menjalankan seluruh suite pengujian query count dan edge cases tanpa data race. Perintah demo menjalankan perbandingan langsung antara kedua strategi. Semua artefak pengujian dan catatan audit terdokumentasi lengkap di direktori lab untuk studi arsitektur lebih lanjut.

Standard library benchmarks provide reliable, zero-dependency validation.

Eksperimen kecil, eliminasi latensi tersembunyi.

N+1 is invisible until it hurts. Batch related queries, don't loop them.

Software Engineering Lab 22 – N+1 Query Problem

internal/blog/models.go • internal/blog/store.go
internal/blog/repository.go • repository_test.go
cmd/demo/main.go • README.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/22-n-plus-one-query-problem`

Lukman / lukman-ss

Sumber: commit 6649acb9
Test passing dan diverifikasi (Race-Clean Go 1.22).
Invarian terbukti: 4 query (N+1) dipangkas menjadi 2 query (Batching).